

Práctica: Sistema de Conteo Automático con Visión Artificial y YOLO para Video

Objetivo: Desarrollar un sistema capaz de detectar y contar objetos utilizando redes neuronales convolucionales (YOLO) y Python.

La librería `ultralytics` está diseñada para que si le pasas un video en lugar de una foto, ella entienda automáticamente que debe procesarlo cuadro por cuadro y guardar el resultado.

```
from ultralytics import YOLO

# 1. Cargar el modelo (usamos el nano 'n' por velocidad, ideal para video)
# Nota: Si 'yolo26n.pt' es hipotético, usa 'yolov8n.pt' o 'yolol1n.pt'
model = YOLO("yolo26n.pt")

# 2. Ejecutar la predicción sobre el archivo de video
# 'save=True' guarda el video resultante con las cajas dibujadas
# 'classes=[2, 3, 5, 7]' filtra para detectar solo vehículos (ver abajo)
results = model.predict(source="video_trafico.mp4", save=True,
                        classes=[2, 3, 5, 7])

print("Video procesado y guardado.")
```

¿Por qué usamos `classes = [2, 3, 5, 7]`?

El modelo preentrenado (COCO) detecta 80 cosas (jirafas, cepillos de dientes, etc.). En tráfico solo nos interesan:

- 2: Car (Coche)
 - 3: Motorcycle (Moto)
 - 5: Bus (Autobús)
 - 7: Truck (Camión)
-

Procesamiento en Bucle con OpenCV

Si quieres **contar cuántos coches pasan** o analizar la velocidad, necesitas abrir el video con OpenCV y pasarle cada imagen a YOLO manualmente.

Aquí tienes el código completo explicado:

```
import cv2
from ultralytics import YOLO

# 1. Cargar modelo
model = YOLO("yolo26n.pt")

# 2. Abrir el video de entrada
video_path = "video_trafico.mp4"
cap = cv2.VideoCapture(video_path)

# (Opcional) Configurar para guardar el video de salida
ancho = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
alto = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))
writer = cv2.VideoWriter('trafficCam.mp4',
cv2.VideoWriter_fourcc(*'mp4v'), fps, (ancho, alto))

# 3. Bucle frame a frame
while cap.isOpened():
    success, frame = cap.read()

    if success:
        # --- APLICAR YOLO ---
        # conf=0.5: Solo objetos con más del 50% de seguridad
        # classes=[2,3,5,7]: Solo vehículos
        results = model(frame, conf=0.5, classes=[2, 3, 5, 7])

        # --- VISUALIZAR ---
        # .plot() dibuja las cajas en el frame automáticamente
        frame_annotado = results[0].plot()

        # (Aquí podrías añadir lógica extra: conteo, líneas virtuales,
        etc.)

        #--- Conteo ---#

        # Guardar en el video de salida
```

```

writer.write(frame_annotado)

# Mostrar en pantalla
cv2.imshow("Deteccion de Trafico", frame_annotado)

# Salir si presionas 'q'
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    break # Fin del video

# 4. Limpiar memoria
cap.release()
writer.release()
cv2.destroyAllWindows()

```

Puntos clave para tráfico:

1. **Modelo Nano (n):** Para tráfico en tiempo real, usa siempre el modelo más pequeño (yolo26n.pt o yolov8n.pt). Los modelos grandes (l o x) son más precisos pero harán que el video vaya muy lento (bajos FPS).
2. **Confianza (conf):** En tráfico, a veces hay reflejos o sombras. Subir la confianza (conf=0.5) ayuda a evitar que detecte "falsos coches" en las sombras.

Entregable

Ahora es tu turno. Utilizando el video de tráfico facilitado, adapta el sistema para detectar y contar exclusivamente vehículos (coches, motos, autobuses y camiones).

Deberás entregar el archivo .ipynb con tu solución y, además, subir un **documento PDF** que documente tu proceso de trabajo.

Para ello, incluye obligatoriamente un apartado de **"Fuentes y Herramientas"** donde detalles qué consultaste en la documentación oficial de YOLO u OpenCV, sobre el código facilitado y el generado por ti, y **qué prompts específicos lanzaste a las IAs** para depurar errores o generar código, analizando las respuestas recibidas y como las has aplicado a tu trabajo.

Enlace al video: [UT03](#)

